

Chapter 1

Monte Carlo Methods

Contents

1	Introduction	2
2	Monte Carlo Prediction	2
3	Monte Carlo Estimation of Action Values	3
4	Monte Carlo Control	3
4.1	Policy Improvement	3
4.2	Algorithms	4
4.2.1	Monte Carlo Control with Exploring Start	4
4.2.2	Monte Carlo Control without Exploring Start	4
4.3	Dealing with Infinite Iterations	6
5	Off-policy Monte Carlo Method	6
5.1	Introduction	6
5.2	Importance Sampling	6
5.3	Off-policy Monte Carlo Prediction	7
5.3.1	Incremental Implementation	7
5.4	Off-policy Monte Carlo Control	8
6	Reducing Variance of Estimators	9
6.1	Discount-aware Importance Sampling	9
6.2	Per-decision Importance Sampling	10

1 Introduction

In this chapter, the Monte Carlo methods are discussed. Unlike DP, Monte Carlo does not require a perfect MDP model. Monte Carlo learns from experience, or simulated experience, if a model is ready.

Monte Carlo methods are ways of solving reinforcement problem based on averaging sample returns. Unfortunately, the Monte Carlo method can be used only on episodic tasks.

2 Monte Carlo Prediction

When a policy is given, we can estimate a state-value function of the policy by **Monte Carlo prediction**. Monte Carlo prediction averages the sample value of desired states and use it as the estimate of state-value function. The averaging is done when each episode terminates, unlike ordinary DP method which updates in each step. This idea of sampling and averaging the returns underlies all Monte Carlo methods.

Precisely, Monte Carlo prediction are divided into two methods; *first-visit MC method* and *every-visit MC method*. The former one estimates $v_\pi(s)$ as the average of the returns first visits to s . The latter one averages the returns following all visits to s . These two MC methods are very similar but have slightly different theoretical properties.

Algorithm 1: First-Visit MC Prediction, for estimating v_π

Input : π a given policy
Output: $V \simeq v_\pi$ an estimate of v_π

```
// initialize
1  $V(s) \in \mathbb{R} \leftarrow$  arbitrary values, for all  $s \in \mathcal{S}$ 
2  $Ret(s) \leftarrow$  empty list, for all  $s \in \mathcal{S}$ 
3 repeat
4   generate an episode following  $\pi$ :  $S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$ 
5    $G \leftarrow 0$ 
6   foreach step of episode,  $t = T - 1, T - 2, \dots, 1, 0$  do
7      $G \leftarrow \gamma G + R_{t+1}$ 
8     // in every-visit MC method, this if statement is ignored
9     if  $S_t$  did not appear in  $S_0, S_1, S_2, \dots, S_{t-1}$  then
10      append  $G$  to  $Ret(s)$ 
11       $V(S_t) \leftarrow average(R(s))$ 
11 until convergence
```

Due to the law of large numbers, the first-visit MC method and every-visit MC method converges to $v_\pi(s)$ as the number of visits to s goes to infinity.

Note that the MC method does not bootstrap; DP method uses values of other states when estimating a value of a given state. MC does not. Due to this property of MC method, MC method is useful when one requires only values of a subset of states.

3 Monte Carlo Estimation of Action Values

If a model is not available, it is particularly useful to estimate action values rather than state values, since we do not know which action gives better return when a model is unavailable. The value we estimate now changes from $v_\pi(s)$ to $q_\pi(s, a)$. We just have to change state to state-action pair in algorithm 1.

The only complication is that many state-action pairs won't be visited. If π is a deterministic policy, we cannot get the values of action rather than $\pi(s)$. The problem of *exploiting and exploring* rises here again. One way to maintain the exploration is to make the episode start with desired state-action pair, and every state-action pair has nonzero probability of being selected. We call this the assumption of *exploring starts*. It is useful, but notice that it cannot be used in all situations; we cannot manually set the start when interacting with real-life environment.

4 Monte Carlo Control

After the evaluation of policy, we can improve our policy, as we did in DP. The difference is that we have action-value function $q_\pi(s, a)$ instead of $v_\pi(s)$. This means that we do not need model to figure out the greedy policy.

4.1 Policy Improvement

For policy improvement, we select a greedy policy with respect to current action-value function $q_\pi(s, a)$.

$$\pi'(s) = \arg \max_a q_\pi(s, a) \tag{1.1}$$

Note that this new policy π' satisfies the conditions of policy improvement theorem;

$$q_\pi(s, \pi'(s)) = \max_a q_\pi(s, a) \tag{1.2}$$

$$\geq q_\pi(s, \pi(s)) \tag{1.3}$$

$$\geq v_\pi(s). \tag{1.4}$$

4.2 Algorithms

4.2.1 Monte Carlo Control with Exploring Start

Algorithm 2: Monte Carlo Control with Exploring Start

```

Input  :  $\pi_0$  ..... initial policy
Output:  $q_*$  ..... optimal action-value function
           $\pi_*$  ..... optimal policy

// initialize
1  $Q(s, a) \in \mathbb{R} \leftarrow$  arbitrary, for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ 
2  $Ret(s, a) \leftarrow$  empty list, for all  $s \in \mathcal{S}$ 
3 repeat
4    $S_0 \sim \mathcal{S}$ , uniformly
5    $A_0 \sim \mathcal{A}(S_0)$ , uniformly
6   Generate an episode starting from  $S_0, A_0$ , following  $\pi$ :  $S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$ 
7    $G \leftarrow 0$ 
8   foreach step of episode,  $t = T - 1, T - 2, \dots, 1, 0$  do
9      $G \leftarrow \gamma G + R_{t+1}$ 
10    // in every-visit MC method, this if statement is ignored
11    if the pair  $S_t, A_t$  did not appear in  $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$  then
12      append  $G$  to  $Ret(S_t, A_t)$ 
13       $Q(S_t, A_t) \leftarrow avg(Ret(S_t, A_t))$ 
14       $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$ 
14 until convergence
15 return  $Q, \pi$ 

```

One can easily show that algorithm 2 converges to optimal policy. If it converges to suboptimal policy, the value function won't be optimal, and it will cause the policy to change.

4.2.2 Monte Carlo Control without Exploring Start

The following algorithm does not use the exploring start method. The algorithm resolves problem of maintaining exploration by using non-deterministic policy. In this algorithm, ϵ -greedy policy is used as the non-deterministic policy.

Before presenting the algorithm, we make a definition of ϵ -soft policy.

Definition 4.2.1 (ϵ -soft Policy)

The ϵ -soft policy π is a policy satisfying following condition;

$$\pi(a|s) \geq \frac{\epsilon}{|\mathcal{A}(s)|}$$

Algorithm 3: On-Policy First-Visit Monte Carlo Control

Input : $\epsilon > 0$ small positive number for probability of choosing non-greedy action
 π initial ϵ -soft policy
Output: q_* optimal action-value function
 π optimal policy

```
// initialize
1  $Q(s, a) \in \mathbb{R} \leftarrow$  arbitrary, for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ 
2  $Ret(s, a) \leftarrow$  empty list, for all  $s \in \mathcal{S}$ 
3 repeat
4   Generate an episode following  $\pi$ :  $S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$ 
5    $G \leftarrow 0$ 
6   foreach step of episode,  $t = T - 1, T - 2, \dots, 1, 0$  do
7      $G \leftarrow \gamma G + R_{t+1}$ 
8     // in every-visit MC method, this if statement is ignored
9     if the pair  $S_t, A_t$  did not appear in  $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$  then
10      append  $G$  to  $Ret(S_t, A_t)$ 
11       $Q(S_t, A_t) \leftarrow avg(Ret(S_t, A_t))$ 
12       $A_* \leftarrow \arg \max_a Q(S_t, a)$ 
13       $\pi(a|S_t) \leftarrow \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(S_t)|} & \text{if } a = A_* \\ \frac{\epsilon}{|\mathcal{A}(S_t)|} & \text{if } a \neq A_* \end{cases}$ 
13 until convergence
14 return  $Q, \pi$ 
```

We can show that this algorithm works well by showing that ϵ -greedy algorithm satisfies conditions of policy improvement theorem over all ϵ -soft policies;

Proof. Let π be any ϵ -soft policy. Let π' be ϵ -greedy policy. Then,

$$q_\pi(s, \pi'(s)) = \sum_a \pi'(a|s) q_\pi(s, a) \tag{1.5}$$

$$= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + (1 - \epsilon) \max_a q_\pi(s, a) \tag{1.6}$$

$$\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + (1 - \epsilon) \sum_a \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1 - \epsilon} q_\pi(s, a) \tag{1.7}$$

$$= v_\pi(s) \tag{1.8}$$

□

Note that we used that the maximum is always bigger than weighted average with weights that sum to 1 in line 1.7. The equality can hold only when π' and π are both optimal policy among the ϵ -soft policies. This can be proved by using the uniqueness of the solution of Bellman optimal equation of state-value function.

4.3 Dealing with Infinite Iterations

The algorithms shown earlier assumes that the loop is repeated infinitely. However, it is impossible to repeat forever in real world. Two methods can be used to relax the infinite loop condition.

1. Hold the idea of approximating q_π ; This idea was used along almost all algorithms we saw.
2. Give up trying to approximate q_π ; This is the idea of GPI. One extreme example of this method is value iteration of DP method. We may not get the precise state value in certain time step, but the final result is still optimal.

5 Off-policy Monte Carlo Method

5.1 Introduction

The dilemma of reinforcement learning is always this: exploit or explore. One of the way of solving this dilemma is to use **off-policy** method.

Off-policy method uses two policy, one which is learned about and becomes optimal policy and one that is more exploratory and us used to generate behavior. The policy being learned is called **target policy**, and the policy used to generate behavior is called **behavior policy**.

To use episodes generated by behavior policy b to estimate values for target policy π , following condition must be satisfied.

Definition 5.1.1 (Assumption of Coverage)

Let π be a target policy and b a behavior policy. Then, the following **assumption of coverage** must be satisfied.

$$\pi(a|s) > 0 \Rightarrow b(a|s) > 0 \quad (1.9)$$

That is, we require that every action taken under π is also taken, at least occasionally, under b .

5.2 Importance Sampling

Almost all off-policy method uses **importance sampling** method. A general introduction to importance sampling method is given in this [document](#). Suppose that we generated an episode starting with state S_t , and got the state-action trajectory $A_t, S_{t+1}, A_{t+1}, \dots, S_T$. Then, the probability of the trajectory occuring under policy π is

$$\mathbb{P}\{A_t, S_{t+1}, A_{t+1}, \dots, S_T | S_t, A_{t:T-1} \sim \pi\} \quad (1.10)$$

$$= \pi(A_t|S_t)p(S_{t+1}|S_t, A_t)\pi(A_{t+1}|S_{t+1}) \cdots p(S_T|S_{T-1}, A_{T-1}) \quad (1.11)$$

$$= \prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k) \quad (1.12)$$

Similarly, the probabilty of the trajectory occuring under policy b is $\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}|S_k, A_k)$. Then, the **importance-sampling ratio** $\rho_{t:T-1}$ is defined as following;

Definition 5.2.1 (Importance-sampling Ratio of Target and Behavior Policy)

Let π and b be the target and behavior policy, respectively. Then, the **importance-sampling ratio** $\rho_{t:T-1}$ is defined as

$$\rho_{t:T-1} = \frac{\prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}|S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)} \quad (1.13)$$

Then, the expected return (values) are calculated by multiplying the $\rho_{t:T-1}$ to the returns G_t due to the policy b .

$$\mathbb{E}[\rho_{t:T-1}G_t|S_t = s] = v_\pi(s) \quad (1.14)$$

5.3 Off-policy Monte Carlo Prediction

Now we apply the off-policy method to Monte Carlo prediction. We define some notations here.

- $\mathfrak{T}(s)$: the set of all time steps in which state s is visited.
- $T(t)$: the first time of termination following time t .

Using these notations, $\{G_t\}_{t \in \mathfrak{T}(s)}$ are the returns that pertain to state s and $\{\rho_{t:T(t)-1}\}_{t \in \mathfrak{T}(s)}$ are the corresponding importance-sampling ratios.

There are two ways of estimating the value. First is the **ordinary importance sampling** and the second is **weighted importance sampling**. The first method estimates value as

$$V(s) = \frac{\sum_{t \in \mathfrak{T}(s)} \rho_{t:T(t)-1} G_t}{|\mathfrak{T}(s)|}. \quad (1.15)$$

The second method estimates value as

$$V(s) = \frac{\sum_{t \in \mathfrak{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in \mathfrak{T}(s)} \rho_{t:T(t)-1}}. \quad (1.16)$$

The ordinary estimator is always v_π in the expectation (unbiased) but can be extreme, and the weighted estimator may produce v_b in the expectation (when only a one sample is observed) but the variance is bounded.

Following is the table of biasedness and unboundedness of variance of each method.

	ordinary	weighted
first-visit	unbiased, unbounded	biased, bounded
every-visit	biased, unbounded	biased, bounded

Since the states in every-visit method is correlated, the estimator is biased, but is asymptotically unbiased as the number of sample grows. In practice, the weighted estimator usually has dramatically lower variance and is strongly preferred.

5.3.1 Incremental Implementation

Incremental implementation can be used in off-policy Monte Carlo Prediction as we did in bandit problem. The difference is that the update occurs episode by episode, not step by step.

The incremental implementation of ordinary importance sampling is simple, since we just have to replace the reward to the weighted return of the algorithm ??.

Weighted importance sampling can also be implemented incrementally. Let $G_1, G_2, \dots, G_{n-1}$ be a sequence of returns, all starting in same state s , each with a corresponding weight $W_i = \rho_{t_i:T(t_i)-1}$ where t_i denotes the time when state s occurred, with return G_i . Let the weighted estimate of value of s be $V_n(s) = \frac{\sum_{k=1}^{n-1} W_k G_k}{\sum_{k=1}^{n-1} W_k}$. The update rule for $V_n(s)$ is

$$V_{n+1}(s) = V_n(s) + \frac{W_n}{C_n} [G_n - V_n] \quad (1.17)$$

where

$$C_{n+1} = C_n + W_{n+1}, \quad (1.18)$$

with $C_0 = 0$.

Algorithm 4: Off-policy Monte Carlo Prediction

Input : π target policy

Output: q_π action-value function of policy π

// initialize

1 **foreach** (s, a) where $s \in \mathcal{S}, a \in \mathcal{A}(s)$ **do**

2 $Q(s, a) \in \mathbb{R} \leftarrow$ arbitrary

3 $C(s, a) \leftarrow 0$

4 **repeat**

5 $b \leftarrow$ any policy satisfying assumption of coverage with π

6 Generate an episode following b : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

7 $G \leftarrow 0$

8 $W \leftarrow 1$

9 **for** $t \in T-1, T-2, \dots, 0$, while $W \neq 0$ **do**

10 $G \leftarrow \gamma G + R_{t+1}$

11 $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

12 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$

13 $W \leftarrow W \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$

14 **until** convergence

15 **return** Q

5.4 Off-policy Monte Carlo Control

Now we present the off-policy Monte Carlo control method. The target policy π and behavior policy b are used, and the assumption of coverage is satisfied. To assure that every possible state-action pair is visited, we set b to be an ε -soft policy. π is a deterministic policy.

Algorithm 5: Off-policy Monte Carlo Control

```
Input  : .....
Output:  $q_*$  ..... optimal action-value function
        $\pi_*$  ..... optimal policy

// initialize
1 foreach  $(s, a)$  where  $s \in \mathcal{S}, a \in \mathcal{A}(s)$  do
2    $Q(s, a) \in \mathbb{R} \leftarrow$  arbitrary
3    $C(s, a) \leftarrow 0$ 
4    $\pi(s) \leftarrow \arg \max_a Q(s, a)$ 
5 repeat
6    $b \leftarrow$  any  $\varepsilon$ -soft policy
7   Generate an episode following  $b$ :  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$ 
8    $G \leftarrow 0$ 
9    $W \leftarrow 1$ 
10  for  $t \in T-1, T-2, \dots, 0$  do
11     $G \leftarrow \gamma G + R_{t+1}$ 
12     $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$ 
13     $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$ 
14     $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$ 
15    if  $A_t \neq \pi(S_t)$  then
16      exit inner loop (proceed to next episode)
      // since the policy  $\pi$  is deterministic,  $\pi(S) \neq A$  leads to zero weight.
17     $W \leftarrow W \frac{1}{b(A_t|S_t)}$ 
18 until convergence
19 return  $Q \simeq q_*, \pi \simeq \pi_*$ 
```

Note that since π is deterministic, the weight W is zero if $A_t \neq \pi(S_t)$ in any time step t . Multiplying $\frac{1}{b(A_t|S_t)}$ instead of $\frac{\pi(A_t|S_t)}{b(A_t|S_t)}$ will seem reasonable now.

6 Reducing Variance of Estimators

6.1 Discount-aware Importance Sampling

The off-policy method we discussed earlier uses importance-sampling, without taking into account the internal structure of return. **Discount-aware importance sampling** uses this structure to reduce the variance of off-policy estimators.

The idea is to think of γ , the discount rate, as the probability of termination. We define *flat partial returns* $\bar{G}_{t:h} =$

$R_{t+1} + R_{t+2} + \dots + R_h$ where $0 \leq t < h \leq T$. h is called *horizon*. Then, the conventional return can be written as

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T \quad (1.19)$$

$$= (1 - \gamma) R_{t+1} \quad (1.20)$$

$$+ (1 - \gamma) \gamma (R_{t+1} + R_{t+2}) \quad (1.21)$$

$$+ (1 - \gamma) \gamma^2 (R_{t+1} + R_{t+2} + R_{t+3}) \quad (1.22)$$

$$\vdots \quad (1.23)$$

$$+ (1 - \gamma) \gamma^{T-t-2} (R_{t+1} + R_{t+2} + \dots + R_{T-1}) \quad (1.24)$$

$$+ \gamma^{T-t-1} (R_{t+1} + R_{t+2} + \dots + R_T) \quad (1.25)$$

$$= (1 - \gamma) \sum_{h=t+1}^{T-1} \gamma^{h-t-1} \bar{G}_{t:h} + \gamma^{T-t-1} \bar{G}_{t:h} \quad (1.26)$$

Now, we define two estimators analogous to ordinary and weighted importance-sampling estimators, with importance-sampling ratio truncated similarly.

$$V(s) = \frac{\sum_{t \in \mathfrak{T}(s)} \left((1 - \gamma) \sum_{h=t+1}^{T(t)-1} \gamma^{h-t-1} \rho_{t:h-1} \bar{G}_{t:h} + \gamma^{T(t)-t-1} \rho_{t:T(t)-1} \bar{G}_{t:T(t)} \right)}{|\mathfrak{T}(s)|} \quad (1.27)$$

$$V(s) = \frac{\sum_{t \in \mathfrak{T}(s)} \left((1 - \gamma) \sum_{h=t+1}^{T(t)-1} \gamma^{h-t-1} \rho_{t:h-1} \bar{G}_{t:h} + \gamma^{T(t)-t-1} \rho_{t:T(t)-1} \bar{G}_{t:T(t)} \right)}{\sum_{t \in \mathfrak{T}(s)} \left((1 - \gamma) \sum_{h=t+1}^{T(t)-1} \gamma^{h-t-1} \rho_{t:h-1} + \gamma^{T(t)-t-1} \rho_{t:T(t)-1} \right)} \quad (1.28)$$

We call these two estimators **discounting-aware importance sampling estimators**.

6.2 Per-decision Importance Sampling

Lets inspect the weighted return $\rho_{t:T-1} G_t$. It can be writte as the sum of rewards multiplied by discount rate

$$\rho_{t:T-1} G_t = \rho_{t:T-1} R_{t+1} + \gamma \rho_{t:T-1} R_{t+2} + \gamma^2 \rho_{t:T-1} R_{t+3} + \dots + \gamma^{T-t-1} \rho_{t:T-1} R_T \quad (1.29)$$

Note that the first term can be written as $\rho_{t:T-1} R_{t+1} = \frac{\pi(A_t|S_t) \dots \pi(A_{T-1}|S_{T-1})}{b(A_t|S_t) \dots b(A_{T-1}|S_{T-1})} R_{t+1}$. The expectations of these factors are all 1.

$$\mathbb{E} \left[\frac{\pi(A_k|S_k)}{b(S_k|A_k)} \right] = 1. \quad (1.30)$$

With more steps, one can show that

$$\mathbb{E}[\rho_{t:T-1} R_{t+k}] = \mathbb{E}[\rho_{t:t+k-1} R_{t+k}]. \quad (1.31)$$

Then defining $\tilde{G}_t = \rho_{t:t} R_{t+1} + \gamma \rho_{t:t+1} R_{t+2} + \gamma^2 \rho_{t:t+2} R_{t+3} + \dots + \gamma^{T-t-1} \rho_{t:T-1} R_T$, it follows that

$$\mathbb{E}[\rho_{t:T-1} G_t] = \mathbb{E}[\tilde{G}_t]. \quad (1.32)$$

We call this **per-decision importance sampling**. The estimator analogous to the ordinary importance sampling estimator is

$$V(s) = \frac{\sum_{t \in \mathfrak{T}(s)} \tilde{G}_t}{|\mathfrak{T}(s)|} \quad (1.33)$$

which we expect to sometime be of lower variance. There is no obvious per-decision version of weighted importance sampling. The estimators proposed up to today are all non-consistent. (They do not converge to true value even with infinite data.)